

2.1 Gradient Descent

2.1.1

1. Descent path differences (step size 0.01): The L2 path is smooth and goes straight to the center. This is because its steps naturally get smaller as it gets closer to zero. The L1 path zig-zags with sharp turns. This happens because its steps always stay the exact same size, no matter how close it is to zero.

2. Two things we see in the L1 path: When using a fixed step size on L1, we see two things:

- Bouncing: Because the steps never get smaller, the path keeps jumping back and forth over the center. It cannot stop exactly at the minimum.
- Straight lines: The path moves in straight, box-like lines along the axes and makes sharp corners.

3. Convergence rate explanation: Finding the goal depends on the step size. For L2, normal step sizes (like 0.1 or 0.01) work well. If the step size is too big (like 1), it jumps too far and fails. If it is too small (like 0.001), it is too slow. For L1, large steps jump too far, and even normal steps will just bounce around the goal at the end.

4. Lowest loss achieved: (Note: These numbers are the best values found during the whole run, not just the last step).

- Lowest L1 loss: 0.0081196196 (with step size 0.01)
- Lowest L2 loss: 0.0000000014 (with step size 0.1)

Why they are different: L2 reaches a much lower loss because its steps naturally get tiny at the end, letting it stop exactly on the target. L1 is stuck taking the same big steps, so it keeps jumping over the lowest point.

2.1.2 Comparing Exponentially Decaying step sizes

5 & 6. Exponential decay vs. Fixed step size: Exponential decay works much better for L1. It starts with big steps to move fast, and then slowly makes the steps smaller. This stops the bouncing at the end. Because the steps get tiny, the model can get much closer to the true minimum.

- Lowest L1 loss with decay: 0.0000000001 (with a decay rate of 0.99).

7. Descent path for a decay rate of 0.95: At first, the path looks just like the fixed step size. But as the steps get smaller over time, the bouncing slows down. Eventually, the path smoothly settles exactly at the center instead of jumping around.

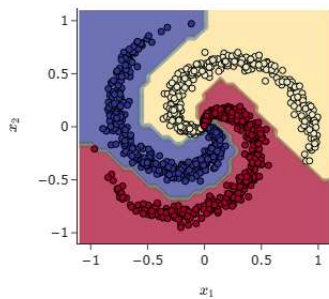
2.2 Neural Networks

We test the model on a spiral dataset with 3 classes. These classes cannot be separated by straight lines.

Q1: Baseline Network We trained a network with two hidden layers (16 neurons each).

- **Accuracy:** 1.0000.
- **Decision Boundary:**

Network Decision Boundaries Q1



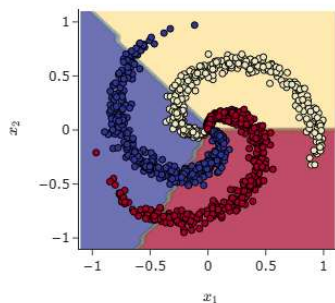
Q2: Network without Hidden Layers

The experiment was repeated with a network containing no hidden layers, which is equivalent to a linear model.

Accuracy: The test accuracy is 0.5367.

Decision Boundary:

Network Decision Boundaries Q2

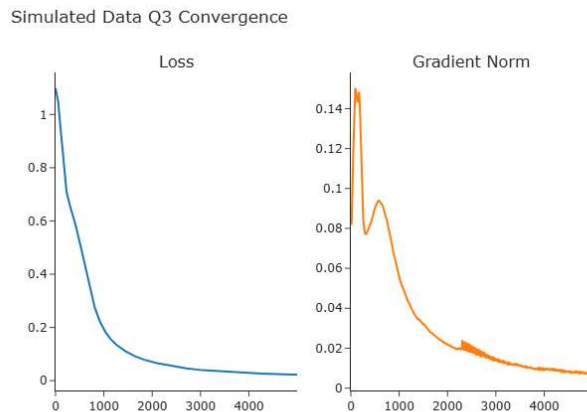


Explanation: The data is designed to be non-linearly separable. A neural network without hidden layers can only learn linear decision boundaries (hyperplanes). As seen in the plot, the model tries to separate the spiral classes with straight lines, which is an impossible task. This lack of model capacity to capture the data's complexity results in very poor performance compared to the non-linear model in Q1.

Q3: Convergence Process of Baseline Network

The convergence of the baseline network from Q1 was tracked by recording the loss and the norm of the gradients at each iteration.

Convergence Plot: The plot below shows the loss and gradient norm as a function of the training iteration.



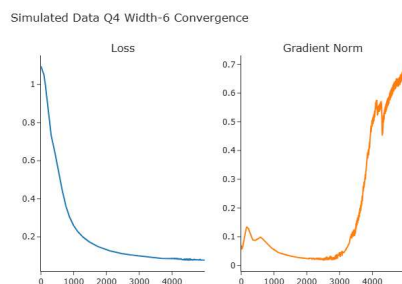
Analysis: Both the loss and the gradient norm decrease steadily over time, indicating that the gradient descent algorithm is successfully minimizing the objective function and the network is learning. The curves eventually flatten out, which suggests the model has converged to a local minimum. The final loss was 0.0229 and the final gradient norm was 0.0072.

Q4: Network with Reduced Hidden Layer Width

The experiment was repeated with a network where the hidden layer width was reduced from 16 to 6 neurons.

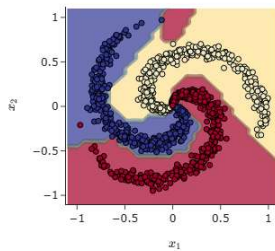
Accuracy: The test accuracy is 0.9767.

Convergence Plot: The convergence process for the 6-neuron network is shown below.



Decision Boundary: The decision boundary for the 6-neuron network is shown below.

Network Decision Boundaries Q4



Explanation: Reducing the number of neurons in the hidden layers reduces the model's capacity, which is its ability to learn complex functions. Compared to the 16-neuron network, the 6-neuron network has a harder time fitting the intricate spiral data, resulting in a lower test accuracy. The decision boundary is visibly less complex and does not separate the classes as cleanly. The convergence plot shows that this simpler model may converge faster (or in fewer iterations), but it converges to a higher final loss value because it lacks the capacity to find a better solution.

MNIST Digit Classification (2.2.2)

The experiments in this section are performed on the MNIST dataset of handwritten digits. The baseline architecture is a network with two hidden layers of 64 neurons each, trained with Stochastic Gradient Descent (SGD).

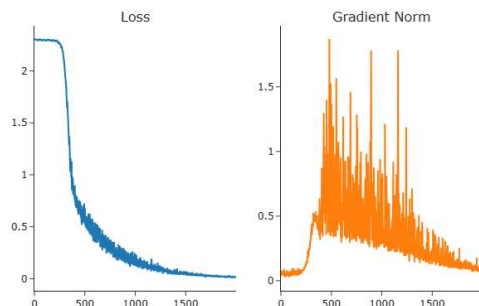
Q5: Baseline Network Test Accuracy

Accuracy: The test accuracy is 0.9736.

Q6: Baseline Network Convergence Process

Convergence Plot: The plot below shows the loss and gradient norm as a function of the training iteration for the baseline network.

MNIST Q6 Convergence

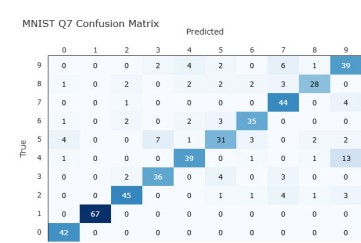


Analysis: The loss and gradient norm show a clear downward trend, but with significant noise and fluctuations. This is characteristic of Stochastic Gradient Descent (SGD), where the gradient is

estimated on a small mini-batch at each step. While noisy, the overall trajectory demonstrates that the network is learning and converging towards a good solution.

Q7: Confusion Matrix Analysis

Confusion Matrix: The confusion matrix below visualizes the performance of the classifier on the test set, showing which digits are most often confused for one another.



Analysis:

Two Most Common Confusions: (True Label, Predicted Label, Count) are [(7, 9, 23), (9, 4, 13)].

Three Least Common Confusions: (True Label, Predicted Label, Count) are [(6, 3, 1), (6, 2, 1), (8, 0, 1)].

Explanation: These results make intuitive sense. The most common confusions occur between digits that are visually similar. A handwritten '4' can easily look like a '9' if the top is closed. A '7' can look like a '9' if it has a curve. Similarly, '3' and '8' or '2' and '7' are often confused. The least common confusions are between digits that have very different structures, like '1' (a vertical line) and '0' (a circle), making them easy for the model to distinguish.

Q8: Network without Hidden Layers

Accuracy: The test accuracy is 0.9230.

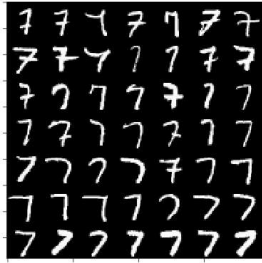
Explanation: The high accuracy (well above 10% for random guessing) suggests that the MNIST digits are, to some extent, linearly separable in the high-dimensional pixel space. A linear model can find hyperplanes that separate many of the digit classes from each other. However, the drop in accuracy from the deep network (Q5) confirms that the relationships are fundamentally non-linear, and hidden layers are crucial for capturing the complex variations in handwriting to achieve state-of-the-art performance.

Q9: Most and Least Confident Predictions for Digit '7'

The model's confidence is the value of the highest probability in the softmax output. We can inspect the images for which the model is most and least confident in its prediction of the digit '7'.

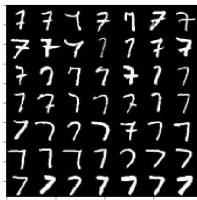
Most Confident '7's:

49 most confident true-7 predictions



Least Confident '7's:

49 least confident true-7 predictions



Confidence Range for true-7 samples: min=0.3622, max=1.0000.

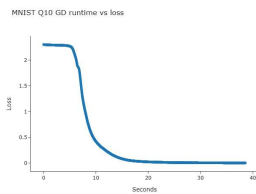
Explanation of Differences:

Both files look the same

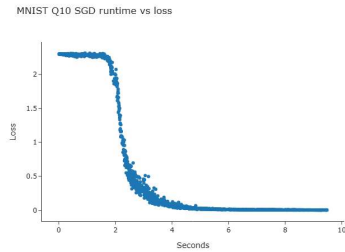
Q10: GD vs. SGD Runtime Comparison

A comparison was made between full-batch Gradient Descent (GD) and mini-batch Stochastic Gradient Descent (SGD) on a subset of the training data.

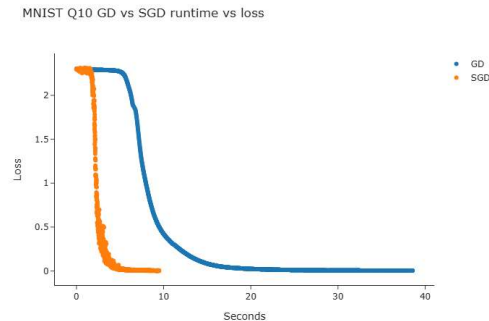
GD Runtime vs. Loss:



SGD Runtime vs. Loss:



Combined GD vs. SGD Plot:



Explanation of Similarities and Differences:

Runtime: The most striking difference is the time per iteration. Each GD iteration is very slow because it processes the entire dataset (2500 samples) to compute the gradient. In contrast, SGD is much faster per iteration, as it only uses a small mini-batch (64 samples). The plots show SGD completing many iterations and reducing loss significantly in the time it takes GD to complete just one or two iterations.

Loss Scale and Curve Shape: The GD loss curve is smooth and decreases monotonically. Each step is guaranteed to improve the loss (with a proper learning rate). The SGD loss curve is very noisy and fluctuates. While the general trend is downward, the loss can temporarily increase. This is because the gradient is only an estimate based on a small batch.

Overall: The combined plot clearly shows the practical advantage of SGD. It makes much faster progress in wall-clock time, reaching a low loss value far quicker than GD. While GD's path is more direct, the computational cost of each step makes it impractical for large datasets, which is why SGD and its variants are the standard for training deep neural networks.

The final loss for GD was 0.0034 after 38.56s.

The final loss for SGD was 0.0030 after 9.48s.

324083245 |^{see} |^{pdf}

I M L - theoretical sec

1.1: 1) $f_1, \dots, f_m: C \rightarrow \mathbb{R}$ convex $\gamma_1, \dots, \gamma_m \in \mathbb{R}_+$

$$\text{קמור} \quad \sum_{i=1}^m \gamma_i f_i(x) = g(x) \quad (3)$$

$x, y \in C, t \in [0, 1]$ ו:

$$\begin{aligned} g(tx + (1-t)y) &= \sum_{i=1}^m \gamma_i f_i(tx + (1-t)y) \leq \sum_{i=1}^m \gamma_i (tf_i(x) + (1-t)f_i(y)) \\ &= \sum_{i=1}^m \gamma_i t f_i(x) + \sum_{i=1}^m \gamma_i (1-t) f_i(y) = t \sum_{i=1}^m \gamma_i f_i(x) + (1-t) \sum_{i=1}^m \gamma_i f_i(y) \\ &= t g(x) + (1-t) g(y) \quad \square \end{aligned}$$

2) קמור $h(x) = -(\lambda^2) = -x^2$ קמור $f(x) = -x$ קמור $g(x) = x^2$

1.2:

$$\text{קמור} \quad \max(0, 1 - \gamma(x^T W + b)) \quad (3)$$

ניגון קמור $1 - \gamma(x^T W + b)$ פונקציה קמורה (היא קמורה) γ קמורה

התוצאה הכוללת $\max(f, g)$ קמורה f, g קמורות

0 פונקציה קמורה וקמור
התוצאה הכוללת.

$$f = 1 - \gamma(x^T W + b), f(w, b) < 0 \text{ כולל}$$

$$\begin{aligned} f &= 1 - \gamma x^T W - \gamma b, \quad \frac{\partial f}{\partial b} = -\gamma \\ &\quad \frac{\partial f}{\partial W} = -\gamma x^T \end{aligned} \Rightarrow \begin{pmatrix} -\gamma \\ -\gamma x^T \end{pmatrix} = \nabla f(w, b)$$

$$\gamma = \begin{cases} \begin{pmatrix} 0 \\ -\gamma \end{pmatrix} & f(w, b) \leq 0 \\ \begin{pmatrix} -\gamma \\ -\gamma x^T \end{pmatrix} & f(w, b) > 0 \end{cases} \Leftrightarrow \nabla f(w, b) = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad \begin{matrix} f(w, b) < 0 \text{ כולל} \\ f(w, b) = 0 \text{ כולל} \end{matrix}$$

$$\forall k \in [m] \quad g_k \in \partial f_k(x) \quad \text{כאשר } f_1, \dots, f_m: \mathbb{R}^d \rightarrow \mathbb{R} \quad (5)$$

$$\forall x \in \mathbb{R}^d \quad \sum_{k=1}^m g_k \in \partial \sum_{k=1}^m f_k(x) \quad \text{כי } f(x) = \sum_{k=1}^m f_k(x)$$

$$f_k(y) \geq f_k(x) + g_k^T(y-x) \quad : (1) \text{ כי } x \rightarrow 1, 0 \text{ ו } y < 1 \text{ אז } (1) \text{ נכונה}$$

$$\sum_{k \in [m]} f_k(y) \geq \sum_{k \in [m]} (f_k(x) + g_k^T(y-x)) = \sum_{k \in [m]} f_k(x) + \sum_{k \in [m]} g_k^T(y-x)$$

$$\therefore \sum_{k \in [m]} f_k(x) \geq \sum_{k \in [m]} g_k^T(y-x) \quad , (1) \text{ כי } x \rightarrow 1, 0 \text{ ו } y < 1 \text{ אז } (1) \text{ נכונה}$$

$$f(w, b): \mathbb{R}^d \rightarrow \mathbb{R} = \frac{1}{m} \sum_{i=1}^m \ell_{x_i, y_i}^{h, w, b}(w, b) \quad S = \{(x_i, y_i)\}_{i=1}^m \subseteq \mathbb{R}^d \times \{\pm 1\} \quad (6)$$

$$\partial_w f(w, b) = \frac{1}{m} \sum_{i=1}^m \left(\begin{cases} (-y_i, x_i^T) & \ell_{x_i, y_i}^{h, w, b}(w, b) > 0 \\ (0) & \text{" } < 0 \end{cases} \right) + \lambda w$$

$$\text{ReLU activation} = \sigma \quad b \in \mathbb{R}^m \quad w \in \mathbb{R}^{m \times d} \quad h = \sigma(w x + b) \quad x \in \mathbb{R}^d \quad (7)$$

$$c \in \mathbb{R} \quad u \in \mathbb{R}^m \quad \hat{y} = u^T h + c$$

$$L = \frac{1}{2} (u^T \cdot \sigma(w x + b) + c - y)^2$$

$$\frac{1}{2} (\hat{y} - y)^2 = L(\hat{y}, y)$$

$$\frac{\partial L}{\partial u} = \sigma(w x + b) (u^T \sigma(w x + b) + c - y) \quad (a)$$

$$\frac{\partial L}{\partial c} = u^T \sigma(w x + b) + c - y$$

$$\frac{\partial L}{\partial w} = (u^T \sigma(w x + b) + c - y) \cdot u^T \sigma(w x + b) \cdot x \quad (b)$$

$$\frac{\partial L}{\partial b} = (u^T \sigma(w x + b) + c - y) \cdot u^T \sigma(w x + b)$$

$$(1) \text{ כי } x \rightarrow 1, 0 \text{ ו } y < 1 \text{ אז } (1) \text{ נכונה}$$

$$L_i(z) = CE(e_i, z) = - \sum_{j=1}^k (e_i)_j \log(z_j) = -\log(z_i)$$

.8

$$J_z(L_i) = \left[\frac{\partial L_i}{\partial z_1}, \dots, \frac{\partial L_i}{\partial z_k} \right]$$

$$\forall j \neq i \quad \frac{\partial L_i}{\partial z_j} = 0, \quad \frac{\partial L_i}{\partial z_i} = -\frac{1}{z_i} \Rightarrow [0, \dots, -\frac{1}{z_i}, 0, \dots, 0]$$

